FRED TALKS

27th October 2026

CONTENTS

Agentic Monitoring
BEN CONGDON · 1924 WORDS

Control the ideas, not the code
ANTIREZ.COM · 1245 WORDS

Every layer of review makes you 10x slower
AVERY PENNARUN · 2841 WORDS

Your Agent is a Distributed System (and fails like one)
MAHESH'S BLOG · 1053 WORDS

Agentic Monitoring

BEN CONGDON · 23 OCT 2026 · SOURCE

A recent Anthropic blog post contained a short paragraph, not at all the point of the post, which has been buzzing around my brain since I read it:

> A new level of operational excellence is possible. Our team uses Claude Tag heavily, which has helped us monitor each component of the system. For example, we created Slack channels where we tasked Claude with being obsessively focused on the health of a single component. We call these Claudes "superfans." We have a scion-daemon superfan, a scion-cache superfan, an Antique merge queue superfan, and so on. They have access to our telemetry and logs, and act like an experienced engineer who has nothing better to do than to refresh charts and look for anything strange. We also have traditional alerts for all of our systems, but our superfans can detect subtle and novel issues that make these alerts seem one-dimensional by comparison.

This is *so obviously* the path forward for software systems operations. I've been thinking about monitoring with agents for the past few weeks, and this paragraph crystallized some of those thoughts.

Modern software systems, like the LLMs that write them, are becoming more "grown" than "designed". Yes, human engineers often still give the high-level design and shape to the system, and yes, software systems (for now) are much more interpretable than LLMs – since you can just sit down and read the code, if you are so inclined. However, with the volume of code being written and shipped increasing exponentially, the amount of actual eyes-on-code knowledge that any human operator of a system has is decreasing.

Systems monitoring was never *easy*. There is a whole industry of observability SaaS companies – Sentry, Datadog, Splunk, Honeycomb, to name a few. Observability is expensive.[1] Doing it well requires a specialized human skill set and relies on building experience operating real systems at scale. One can really only develop these skills by working in industry. This is a domain that takes years to train junior engineers on.

As with seemingly everything in the software industry in 2026, system monitoring is poised for a step change due to agents. I… am surprised I have not seen more people write about this, especially in the current milieu of "agent frenzy". It's right there in front of us all, and it *works*. There have been a bunch of startups trying to do the "AI SRE", but very little writing from practitioners.

But you don’t really need a full “AI SRE” to improve operational excellence, nor do you need a built-out SaaS product to do so. As of the Opus 5.5 / Astra class of models, with a bit of care, it’s entirely possible to make a good agent-driven monitoring system that “just works”.

Case study: change-point monitoring

I’ll ground this discussion in a problem that is near to my heart: production monitoring of change points, like binary releases and config pushes. The idea is that you want to do specialized, intensive monitoring around critical points of change to a system; these change points are more likely to trigger a regression, and thus if you monitor these events closely, you can limit its impact. To quote the clichéd line from the Google SRE book, “~70% of outages are due to changes in a live system”.

This is a wicked problem. I’ve been working on it in some form since 2023, and it’s been hard to get traction on. From talking with folks across the industry, I’ve learned there are ways to do change-point monitoring “well”, but these methods largely still rely on humans to do the right thing (e.g. tagging metrics/alerts diligently, defining precise signals, etc.) and require significant organizational investment.

More specifically, achieving “takeoff” for change-point monitoring as a piece of platform engineering is hard because:

- Your first instinct may be to target highly motivated users who already have a good ops posture. However, these teams’ rollouts are already mostly safe, and you don’t actually raise the ops waterline much. The thing you’re selling here is *toil reduction*, which is “nice to have” but is not itself a first-order safety improvement. Precision is easier to achieve for highly motivated teams, but this means the change-point system is “competing” on recall and latency with traditional alerts.

- You can alternatively target the broad population of services. In other words, you go super wide and opt in “all services”, on a chosen subset of universal health metrics. In practice, most of these general health metrics are irreducibly noisy. This noise results in poor precision and alert fatigue.

- Prior to LLMs, there was a clear precision/latency tradeoff as well. Along this spectrum are: structured logs (precise, slow), metrics (coarse, fast), and alerts (even coarser, recall is pretty bad and precision is pretty bad, but “at least it’s something”).

Shockingly, agents largely "just" solve this problem. Agents are happy to run copious queries, can write per-change queries on the fly, and will scan logs ad nauseam. As a result, you can get both good precision and recall for cheap. In the basic version of this setup, you just need the ability to connect your codebase (e.g. to reason about config consumption) and your observability stack (e.g. to view and monitor the live production state) to the agent.

I've spiked a prototype of exactly this: an agent that proposes a monitoring plan based on the service code, a scaffold that executes that plan, and an agent that investigates triggered monitors, with an escalation path to humans. Even in this early prototype, I've been able to detect regressions in systems I have *zero* understanding of with precision and latency that would have been impossible a year ago.

In my iteration on this, I've found that a pure agent loop is not the optimal shape for agentic monitors. Rather, a harness that allows the agent to write a spec of how to monitor, along with a deterministic system for running those specs, produces significantly cleaner signals with dramatically lower token usage. To the extent there is "secret sauce" in this pattern, it's in the noise-reduction heuristics, the prompts that get the agent to write good specs, and a labeled eval set large enough to validate further tweaks.[2]

Agents are structurally well suited to monitoring

Two mindset shifts need to happen before this pattern can catch on:

1. Agents actually can do a much better job at monitoring than humans.
2. Monitoring agents need scaffolding, the same way the rest of the software development lifecycle had to adapt for coding agents. There hasn't been a "Claude Code moment" for monitoring yet.

Once you make these shifts, monitoring agents have several "unfair advantages" over human operators:

- Ability to track everything, 24/7. If tokens are cheap, then any regression in a metric you care about and any suspicious log line can be investigated – not just the events that would previously have risen above the "human attention cutline". Agents don't need to "follow the sun".
- Ability to fan out and investigate several paths toward resolving a production problem. Agents can parallelize investigations when multiple things go wrong at once, and coordinate quickly. This is already my go-to for incident triage: I just launch a bunch of agents to

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name "delete-everything". Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs… but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

investigate several potential threads or theorized incident causes. Right now, this is human-driven, but there's really no reason it couldn't be agent-initiated in the future. You can elastically scale up a fleet of incident responders.
    - We're already seeing that this is becoming critical for security incidents. Teams/fleets/swarms of agents are being used for offensive cyberattacks, and defenders are beginning to build up similar defensive capabilities (e.g. OpenAI's Daybreak Blue, Anthropic's Project Glasswing, Google's Fairwind).

- Ability to respond faster to a problem than can a human. Agents can read code faster than humans. Agents can also query and digest metrics and log signals significantly faster. This speed doesn't always translate into faster incident detection time; a human with a deep understanding of a system may still be faster at finding the right signal and intuiting the root causes. But to the extent that tacit knowledge can be recorded and made available to an agent, the agent will eventually beat the human on speed.
    - By extension, for systems where the human operator isn't deeply experienced, the "time to understanding" is going to be shorter for an agent.

"A new level of operational excellence is possible."

Be more diligent: You wouldn't pay a full-time software engineer to look through all the Git commits for a service, pick out things to monitor, and make special short-term fast-trigger alerts just on those changes. However, for a couple cents per million tokens, it's possible now, and will become an industry standard.

Continuously improve performance: Agents can monitor performance, look at heap dumps and stack traces and CPU profiles, and propose optimizations from the patterns they find. Put the proposed improvements behind A/B tests if necessary, and monitor as you would any other change to the service.

In one instance, I observed that a few sessions of Astra/Opus investigation discovered 50% heap savings and 30% latency reduction in a common hot-path library that humans had already spent nontrivial effort to optimize. Once you give a sufficiently powerful LLM a VM and benchmarks to run, the "hill-climbing" ability of current-generation LLMs is remarkable.

Take action: Thus far, I've mostly been discussing "read-only" monitoring systems. Agents, of course, needn't be "read-only". With the right harness and guardrails in place, there's definitely a future in which agents will be able to take front-line actions to remediate incidents. This could span anything from well-scoped operational tasks like resizing instances or rotating compute pools, to deterministic actions like rolling back a config change or binary, to open-ended tasks like root-causing bugs, proposing fixes, and redeploying on-the-fly. Obviously there is risk the

more control you hand-over to the agent, but considering how much agent capabilities expanded over the course of the past 12 months, the amount of write-path work that can be responsibly handed over to agents is bound to increase. The promise of truly self-healing systems is exciting.

Cost is no longer a limitation. Luna/Haiku-class models[3] are now competent graph-starers. A moderately large tech company could likely monitor its entire fleet for on the order of $100/day. Startup-scale fleets would be a small fraction of that. Compared to human labor, or even the cost of the infra that runs observability, this is trivial.

Create a virtuous cycle: If done well, agentic monitoring creates a positive feedback loop: coding agents allow you to ship more, monitoring agents allow you to support more systems, which in turn frees up human time to direct more things to ship. Humans increasingly spend time "building the factory" or "tending the garden".

Agentic monitoring is worth investing in. It will soon be reckless not to have agents monitoring your systems. Until the rest of the industry catches up, getting started now affords an unfair advantage.

1. A set of optimizations my team recently completed saved us several million dollars in annualized observability cost, and… that was a mere fraction of what *just my team* spends on observability. ↩

2. Much, much more could be written about the design of agentic monitoring harnesses. This is one of the clearest "we're so early" feelings I've had in a while. General harnesses, like Anthropic's "Claude Tag", evidently already do a good job at agentic monitoring, per the original post. However, based on what I've seen, even a little monitoring-specific harness engineering picks up a lot of low-hanging fruit. ↩

3. Here I mean GPT-6-Luna and Claude Haiku 5.5. ↩

Control the ideas, not the code

ANTIREZ.COM · 18 JUL 2026 · SOURCE

```
Look at the past history of this blog. There are many blog posts abou

So mine is a trick. People feel more and more programming is complete
```

Systems design 3: LLMs and the semantic revolution (2025)

Unrelated

SimSWE part 2: The perils of multitasking (2017)

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · SOURCE

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments…). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

It’s a lot easier to build those individual beautiful things in small teams that trust each other, that know what quality looks like *to them*. They deliver their things to customer teams who can clearly explain what quality looks like *to them*. And on we go. Quality starts bottom-up, and spreads.

I think small startups are going to do really well in this new world, probably better than ever. Startups already have fewer layers of review just because they have fewer people. Some startups will figure out how to produce high quality components quickly; others won't and will fail. Quality by natural selection?

Bigger companies are gonna have a harder time, because their slow review systems are baked in, and deleting them would cause complete chaos.

But, it’s not just about company size. I think engineering teams at any company can get smaller, and have better defined interfaces between them.

Maybe you could have multiple teams inside a company competing to deliver the same component. Each one is just a few people and a few coding bots. Try it 100 ways and see who comes up with the best one. Again, quality by evolution. Code is cheap but good ideas are not. But now you can try out new ideas faster than ever.

Maybe we’ll see a new optimal point on the monoliths-microservices continuum. Microservices got a bad name because they were too micro; in the original terminology, a “micro” service was exactly the right size for a “two pizza team” to build and operate on their own. With AI, maybe it's one pizza and some tokens.

What’s fun is you can also use this new, faster coding to experiment with different module *boundaries* faster. *Features* are still hard for lots of reasons, but refactoring and automated integration testing are things the AIs excel at. Try splitting out a module you were afraid to split out before. Maybe it'll add some lines of code. But suddenly lines of code are cheap, compared to the coordination overhead of a bigger team maintaining both parts.

Every team has some monoliths that are a little too big, and too many layers of reviews. Maybe we won't get all the way to Singularity. But, we can engineer a much better world. Our problems are solvable.

It just takes trust.

Related

Highlights on "quality," and Deming's work as it applies to software development (2016)

```
This is why yesterday, on X, I said that I believe many programmers a

1. You can now generate a lot of code, even *not* accounting for the

2. LLMs are very good at writing locally optimal code, and are worse

3. The working day is 8 hours. If you read the code, it is a tradeoff

Controlling the ideas. Do you remember this phrasing from the Mythica

Matteo Collina yesterday asked me, in reply to my tweet: but didn’t y

Fable and GPT 5.6 reviews to the sorted sets memory saving are going
```

:

Every layer of review makes you 10x slower

AVERY PENNARUN · 26 MAR 2026 · SOURCE

2026-03-16 »

Every layer of review makes you 10x slower

We’ve all heard of those network effect laws: the value of a network goes up with the square of the number of members. Or the cost of communication goes up with the square of the number of members, or maybe it was n log n, or something like that, depending how you arrange the members. Anyway doubling a team doesn't double its speed; there’s coordination overhead. Exactly how much overhead depends on how badly you botch the org design.

But there’s one rule of thumb that someone showed me decades ago, that has stuck with me ever since, because of how annoyingly true it is. The rule is annoying because it doesn’t *seem* like it should be true. There’s no theoretical basis for this claim that I’ve ever heard. And yet, every time I look for it, there it is.

Here we go:

Every layer of approval makes a process 10x slower

I know what you're thinking. Come on, 10x? That's a lot. It's unfathomable. Surely we're exaggerating.

Nope.

Just to be clear, we're counting "wall clock time" here rather than effort. Almost all the extra time is spent sitting and waiting.

Look:

- Code a simple bug fix
  30 minutes

- Get it code reviewed by the peer next to you
  300 minutes → 5 hours → half a day

- Get a design doc approved by your architects team first
  50 hours → about a week

- Get it on some other team's calendar to do all that
  (for example, if a customer requests a feature)
  500 hours → 12 weeks → one fiscal quarter

I wish I could tell you that the next step up — 10 quarters or about 2.5 years — was too crazy to contemplate, but no. That's the life of an executive sitting above a medium-sized team; I bump into it all the time even at a relatively small company like Tailscale if I want to change product direction. (And execs sitting above large teams can't actually do work of their own at all. That's another story.)

AI can't fix this

First of all, this isn't a post about AI, because AI's direct impact on this problem is minimal. Okay, so Claude can code it in 3 minutes instead of 30? That's super, Claude, great work.

Now you either get to spend 27 minutes reviewing the code yourself in a back-and-forth loop with the AI (this is actually kinda fun); or you save 27 minutes and submit unverified code to the code reviewer, who will still take 5 hours like before, but who will now be mad that you're making them read the slop that you were too lazy to read yourself. Little of value was gained.

The job of a code reviewer isn't to review code. It's to figure out how to obsolete their code review comment, that whole class of comment, in all future cases, until you don't need their reviews at all anymore.

(Think of the people who first created "go fmt" and how many stupid code review comments about whitespace are gone forever. Now that's engineering.)

By the time your review catches a mistake, the mistake has already been made. The root cause happened already. You're too late.

Modularity

I wish I could tell you I had all the answers. Actually I don't have much. If I did, I'd be first in line for the Singularity because it sounds kind of awesome.

I think we're going to be stuck with these systems pipeline problems for a long time. Review pipelines — layers of QA — don't work. Instead, they make you slower while hiding root causes. Hiding causes makes them harder to fix.

But, the call of AI coding is strong. That first, fast step in the pipeline is *so fast!* It really does feel like having super powers. I want more super powers. What are we going to do about it?

Maybe we finally have a compelling enough excuse to fix the 20 years of problems hidden by code review culture, and replace it with a real culture of quality.

I think the optimists have half of the right idea. Reducing review stages, even to an uncomfortable degree, is going to be needed. But you can't just reduce review stages without something to replace them. That way lies the Ford Pinto or any recent Boeing aircraft.

The complete package, the table flip, was what Deming brought to manufacturing. You can't half-adopt a "total quality" system. You need to eliminate the reviews *and* obsolete them, in one step.

How? You can fully adopt the new system, in small bites. What if some components of your system can be built the new way? Imagine an old-school U.S. auto manufacturer buying parts from Japanese suppliers; wow, these parts are so well made! Now I can start removing QA steps elsewhere because I can just assume the parts are going to work, and my job of "assemble a bigger widget from the parts" has a ton of its complexity removed.

I like this view. I've always liked small beautiful things, that's my own bias. But, you can assemble big beautiful things from small beautiful things.

To say nothing of a whole engineering redesign to improve quality, that would be super expensive and we could be designing all new widgets instead.

Sound like any engineering departments you know?

Well, this isn't the right time to rehash Deming, but suffice it to say, he was on to something. And his techniques worked. You get things like the famous Toyota Production System where they eliminated the QA phase entirely, but gave everybody an "oh crap, stop the line, I found a defect!" button.

Famously, US auto manufacturers tried to adopt the same system by installing the same "stop the line" buttons. Of course, nobody pushed those buttons. They were afraid of getting fired.

Trust

The basis of the Japanese system that worked, and the missing part of the American system that didn't, is trust. Trust among individuals that your boss Really Truly Actually wants to know about every defect, and wants you to stop the line when you find one. Trust among managers that executives were serious about quality. Trust among executives that individuals, given a system that can work and has the right incentives, will produce quality work and spot their own defects, and push the stop button when they need to push it.

But, one more thing: trust that the system *actually does work*. So first you need a system that will work.

Fallibility

AI coders are fallible; they write bad code, often. In this way, they are just like human programmers.

Deming's approach to manufacturing didn't have any magic bullets. Alas, you can't just follow his ten-step process and immediately get higher quality engineering. The secret is, you have to get your engineers to engineer higher quality into the whole system, from top to bottom, repeatedly. Continuously.

Every time something goes wrong, you have to ask, "How did this happen?" and then do a whole post-mortem and the Five Whys (or however many Whys are in fashion nowadays) and fix the underlying Root Causes so that it doesn't happen again. "The coder did it wrong" is never a root cause, only a symptom. Why was it possible for the coder to get it wrong?

Now now, you say, that's not the value of agentic coding. You don't use an agent on a 30-minute fix. You use it on a monstrosity week-long project that you and Claude can now do in a couple of hours! Now we're talking. Except no, because the monstrosity is so big that your reviewer will be extra mad that you didn't read it yourself, and it's too big to review in one chunk so you have to slice it into new bite-sized chunks, each with a 5-hour review cycle. And there's no design doc so there's no intentional architecture, so eventually someone's going to push back on that and here we go with the design doc review meeting, and now your monstrosity week-long project that you did in two hours is... oh. A week, again.

I guess I could have called this post Systems Design 4 (or 5, or whatever I'm up to now, who knows, I'm writing this on a plane with no wifi) because yeah, you guessed it. It's Systems Design time again.

The only way to sustainably go faster is fewer reviews

It's funny, everyone has been predicting the Singularity for decades now. The premise is we build systems that are so smart that they themselves can build the next system that is even smarter, that builds the next smarter one, and so on, and once we get that started, if they keep getting smarter faster enough, then the incremental time (t) to achieve a unit (u) of improvement goes to zero, so (u/t) goes to infinity and foom.

Anyway, I have never believed in this theory for the simple reason we outlined above: the majority of time needed to get anything done is not actually the time doing it. It's wall clock time. Waiting. Latency.

And you can't overcome latency with brute force.

I know you want to. I know many of you now work at companies where the business model kinda depends on doing exactly that.

Sorry.

But you can't just *not review* things!

Ah, well, no, actually yeah. You really can't.

There are now many people who have seen the symptom: the start of the pipeline (AI generated code) is so much faster, but all the subsequent stages (reviews) are too slow! And so they intuit the obvious solution: stop reviewing then!

The result might be slop, but if the slop is 100x cheaper, then it only needs to deliver 1% of the value per unit and it's still a fair trade. And if your value per unit is even a mere 2% of what it used to be, you've doubled your returns! Amazing.

There are some pretty dumb assumptions underlying that theory; you can imagine them for yourself. Suffice it to say that this produces what I will call the AI Developer's Descent Into Madness:

1. Whoa, I produced this prototype so fast! I have super powers!
2. This prototype is getting buggy. I'll tell the AI to fix the bugs.
3. Hmm, every change now causes as many new bugs as it fixes.
4. Aha! But if I have an AI agent also review the code, it can find its own bugs!
5. Wait, why am I personally passing data back and forth between agents
6. I need an agent framework
7. I can have my agent write an agent framework!
8. Return to step 1

It's actually alarming how many friends and respected peers I've lost to this cycle already. Claude Code only got good maybe a few months ago, so this only recenlty started happening, so I assume they will emerge from the spiral eventually. I mean, I hope they will. We have no way of knowing.

Why we review

Anyway we know our symptom: the pipeline gets jammed up because of too much new code spewed into it at step 1. But what's the root cause of the clog? Why doesn't the pipeline go faster?

I said above that this isn't an article about AI. Clearly I'm failing at that so far, but let's bring it back to humans. It goes back to the annoyingly true observation I started with: every layer of review is 10x slower. As a society, we know this. Maybe you haven't seen it before now. But trust me: people who do org design for a living know that layers are expensive... and they still do it.

As companies grow, they all end up with more and more layers of collaboration, review, and management. Why? Because otherwise mistakes get made, and mistakes are increasingly expensive at scale. The average value added by a new feature eventually becomes lower than the average value lost through the new bugs it causes. So, lacking a way to make features produce more value (wouldn't that be nice!), we try to at least reduce the damage.

The more checks and controls we put in place, the slower we go, but the more monotonically the quality increases. And isn't that the basis of *continuous improvement?*

Well, sort of. Monotonically increasing quality is on the right track. But "more checks and controls" went off the rails. That's *only one way* to improve quality, and it's a fraught one.

"Quality Assurance" reduces quality

I wrote a few years ago about W. E. Deming and the "new" philosophy around quality that he popularized in Japanese auto manufacturing. (Eventually U.S. auto manufacturers more or less got the idea. So far the software industry hasn't.)

One of the effects he highlighted was the problem of a "QA" pass in a factory: build widgets, have an inspection/QA phase, reject widgets that fail QA. Of course, your inspectors probably miss some of the failures, so when in doubt, add a second QA phase after the first to catch the remaining ones, and so on.

In a simplistic mathematical model this seems to make sense. (For example, if every QA pass catches 90% of defects, then after two QA passes you've reduced the number of defects by 100x. How awesome is that?)

But in the reality of agentic humans, it's not so simple. First of all, the incentives get weird. The second QA team basically serves to evaluate how well the first QA team is doing; if the first QA team keeps missing defects, fire them. Now, that second QA team has little incentive to produce that outcome for their friends. So maybe they don't look too hard; after all, the first QA team missed the defect, it's not unreasonable that we might miss it too.

Furthermore, the first QA team knows there is a second QA team to catch any defects; if I don't work too hard today, surely the second team will pick up the slack. That's why they're there!

Also, the team making the widgets in the first place doesn't check their work too carefully; that's what the QA team is for! Why would I slow down the production of every widget by being careful, at a cost of say 20% more time, when there are only 10 defects in 100 and I can just eliminate them at the next step for only a 10% waste overhead? It only makes sense. Plus they'll fire me if I go 20% slower.